Blatt 04: Lexer und Parser selbst implementiert

Carsten Gips, BC George (HSBI)

1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung einer Grammatik sowie daraus ableitend die manuelle Implementierung eines Lexers und Parsers mit *recursive descent* für eine Lisp-artige Sprache.

2 Methodik

Sie finden nachfolgend eine Beschreibung der Zielsprache für dieses Blatt.

Erstellen Sie zunächst eine Grammatik für diese Sprache. Implementieren Sie dann basierend auf dieser Grammatik einen Lexer und einen *recursive descent* Parser (manuelle Implementierung, kein ANTLR). Überlegen Sie sich, welche Strukturen ein AST für diese Sprache haben sollte und implementieren Sie diese Strukturen in Java und schreiben Sie eine Transformation des Parse-Trees in diesen AST.

Es ist empfehlenswert, die Implementierung in Java mehrstufig zu realisieren:

1. Einlesen aus einer Datei mit Lisp-artigem Code
2. Lexen des eingelesenen Code
3. Parsen des Tokenstroms
4. Erzeugen und Ausgeben des AST

3 Sprachdefinition

Ein Programm besteht aus einem oder mehreren Ausdrücken (*Expressions*). Die Ausdrücke haben eine spezielle Form: Sie sind sogenannte S-Expressions. Dies sind entweder Literale der Form `x` oder einfache listenartige Gebilde der Form `(. x y)`, wobei der `.` eine Operation (oder Funktion) darstellt und `x` und `y` selbst wieder S-Expressions sind.

Die einfachste Form sind dabei Literale mit konkreten Werten der drei Datentypen `Integer`, `String` und `Boolean`:

```
42          ;; Integer
"hello"     ;; String
true        ;; Boolean
false       ;; Boolean
```

Für eine Variable `foo` wäre das Folgende ebenfalls eine S-Expression:

```
foo         ;; Variable foo
```

(Über `;;` wird ein Kommentar eingeleitet, der bis zum Ende der Zeile geht.)

Komplexere Ausdrücke werden über die Listenform gebildet:

```
(+ 1 1)             ;; 1 + 1
(/ 10 3)            ;; 10 / 3
(+ 1 2 3 4)         ;; 1 + 2 + 3 + 4
```

```
(+ (+ (+ 1 2) 3) 4)    ;; (((1 + 2) + 3) + 4)
(/ (+ 10 2) (+ 2 4))   ;; ((10 + 2) / (2 + 4))
```

In der listenartigen Form ist der erste Eintrag der Liste immer eine Operation (oder ein Funktionsname), danach kommen je nach Operation/Funktion (die Arität muss passen!) entsprechende Einträge, die als Parameter für die Operation oder Funktion zu verstehen sind.

Die Ausdrücke sind implizit von links nach rechts geklammert, d.h. der Ausdruck `(+ 1 2 3 4)` ist *syntactic sugar* für `(+ (+ (+ 1 2)3)4)`.

3.1 Eingebaute Funktionen

Es gibt zwei Funktionen, die fest in der Sprache integriert sind.

Mit der eingebauten Funktion `print` kann der Wert eines Ausdrucks auf der Konsole ausgegeben werden:

```
(print "hello world")
(print "wuppie\nfluppie\nfoo\nbar")
```

Die eingebaute Funktion `str` verknüpft ihre Argumente und bildet einen String. Falls nötig, werden die Argumente vorher in einen String umgewandelt.

```
(str 42)                              ;; liefert "42" zurück
(str "wuppie" "fluppie" "foo" "bar")  ;; liefert "wuppiefluppiefoobar" zurück
(str "one: " 1 ", two: " 2)           ;; liefert "one: 1, two: 2" zurück
```

3.2 Operatoren

Es gibt nur wenige vordefinierte Operatoren, diese mit der üblichen Semantik.

3.2.1 Vergleichsoperatoren

Operation	Operator
Gleichheit	=
Größer	>
Kleiner	<

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist immer vom Typ `Boolean`.

3.2.2 Arithmetische Operatoren

Operation	Operator
Addition	+
Subtraktion	−
Multiplikation	*
Division	/

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist vom Typ der Operanden.

3.3 Kontrollstrukturen (If-Else)

Die **`if`**`-then-`**`else`**-Abfrage gibt es mit und ohne den **`else`**-Zweig:

```
(if boolean-form
    then-form
    optional-else-form)
```

```
(if (< 1 2)
    (print "true")     ;; then
    (print "false"))   ;; else
```

Dabei kann jeweils nur genau eine S-Expression genutzt werden. Wenn man mehrere Dinge berechnen möchte, nutzt man **`do`**:

```
(do
    (print "wuppie")
    (print "fluppie")
    (print "foo")
    (print "bar"))
```

Beispiel:

```
(if (< 1 2) (do (print "true") (print "WUPPIE")) (print "false"))
```

oder anders formatiert:

```
(if (< 1 2)
    (do (print "true")
        (print "WUPPIE"))
    (print "false"))
```

3.4 Variablen: Bindings mit *def* anlegen

```
(def x 42)  ;; definiert eine neue Variable mit dem Namen "x" und dem Wert 42

x           ;; liefert 42
(+ x 7)     ;; liefert 49
```

3.5 Funktionen mit *defn* definieren

```
;;     name   params  body
(defn  hello  (n)     (str "hello " n))  ;; Definition einer Funktion "hello"
    mit einem Parameter

(hello "world")                          ;; Aufruf der Funktion "hello" mit dem
     Argument "world"
```

3.6 Lokale Scopes mit *let*

```
;;    bindings      use names here
(let  (name value)  (code that uses name))

(def x 99)   ;; globale Variable x
(def y 101)  ;; globale Variable y
(def z 42)   ;; globale Variable z
(let (x 1   ;; lokales x mit Wert 1(verdeckt globales x)
      y 2)  ;; lokales y mit Wert 2
     (+ x y z))  ;; 1+2+42

(defn  hello
       (n)
       (let (l 42)  ;; l is valid in this scope
            (str "hello " n ": " l)
       )  ;; end of local scope
)  ;; end of function definition
```

Mit `let` können lokale Variablen erzeugt werden, die dann in dem jeweiligen Scope genutzt werden können. Dies funktioniert wie in anderen Sprachen mit Scopes.

3.7 Rekursion

```
(defn fac (n)
    (if (< n 2)
        1
        (* n (fac (- n 1)))))
```

Da es kein **`while`** oder **`for`** gibt, müssen Schleifen über rekursive Aufrufe abgebildet werden.

3.8 Datenstrukturen

In unserer Sprache gibt es Listen:

```
(1 2 3)          ;; Fehler!
(def v (1 2 3))  ;; Fehler!
```

Das Problem daran ist, dass unsere S-Expressions zwar bereits listenartige Strukturen sind, der erste Eintrag aber als Operator oder Funktion interpretiert wird. Der Ausdruck oben würde beim Auswerten versuchen, die "Funktion" 1 auf den Argumenten 2 und 3 aufzurufen …

Man braucht also eine Notation, die ein sofortiges Auswerten verhindert und nur die Liste an sich zurückliefert. Dies erreicht man durch die Funktion `list`:

```
(list 1 2 3)          ;; (1 2 3)

(def v (list 1 2 3))  ;; v = (1 2 3)
v                     ;; (1 2 3)
```

Mit der Funktion `nth` kann man auf das n-te Element einer Liste zugreifen:

```
(nth (list "abc" false 99) 2)  ;; 99
```

Zusätzlich gibt es die beiden Funktionen `head` und `tail`, die das erste Element einer Liste bzw. die restliche Liste ohne das erste Element zurückliefern:

```
(head (list 1 2 3))  ;; 1
(tail (list 1 2 3))  ;; (2 3)
```

4 Aufgaben

4.1 A4.1: Kontextfreie Grammatik (1P)

Betrachten sie die folgende Grammatik:

$$G = (\{S, A\}, \{1, 2, 3\}, P, S)$$

mit

$$\begin{aligned} P = \{ & \\ & S \quad \to 1AS \mid 3 \\ & A \quad \to 2AS \mid \epsilon \\ \} & \end{aligned}$$

Berechnen die die *First*- und *Follow-Mengen* der Grammatik.

Zeigen Sie, dass die Grammatik LL(1) ist.

4.2 A4.2: Grammatik (2P)

1. Erstellen Sie zunächst einige Programme in der Zielsprache. Diese sollten von einfachsten Ausdrücken bis hin zu komplexeren Programmen reichen. Definieren Sie beispielsweise eine Funktion, die rekursiv die Länge einer Liste berechnet.

   Definieren Sie neben gültigen Programmen auch solche, die in der syntaktischen Analyse zurückgewiesen werden sollten. Welche Fehlerkategorien könnte es hier geben?

2. Definieren Sie nun für die obige Sprache eine geeignete Grammatik.

4.3 A4.3: Lexer (2P)

1. Definieren Sie in Java Strukturen, die Sie für die Repräsentation der Token entsprechend Ihrer Grammatik benötigen.
2. Implementieren Sie dann analog zum Vorgehen in der Vorlesung einen Lexer, der den entsprechenden Teil Ihrer Grammatik abbildet. Diesen Lexer sollen Sie manuell in Java implementieren, Sie dürfen also nicht ANTLR oder andere Generatoren benutzen.

Implementieren Sie dabei das Verarbeiten des Lisp-artigen Codes aus einem übergebenen String.

Sie können wie in der Vorlesung demonstriert auf Anforderung das nächste Token berechnen. Sie können aber auch den Eingabecode vollständig verarbeiten und dann einen Tokenstrom geeignet zurückgeben.

Ihr Lexer soll eine rudimentäre Fehlerbehandlung aufweisen und bei Problemen die Abweichung vom erwarteten Zeichen zum tatsächlich eingelesenen Zeichen ausgeben. Darüber hinaus braucht der Lexer keinen Recovery-Modus haben o.ä., d.h. nach der Fehlermeldung darf Ihr Lexer “aussteigen”.

4.4 A4.4: Parser mit *recursive descent* (3P)

Implementieren Sie analog zum Vorgehen in der Vorlesung einen Parser mit *recursive descent*, der den entsprechenden Teil Ihrer Grammatik abbildet. Diesen Parser sollen Sie manuell in Java implementieren, Sie dürfen also nicht ANTLR oder andere Generatoren benutzen.

Implementieren Sie dabei das Verarbeiten des Lisp-artigen Codes aus einem übergebenen String.

Ihr Parser soll eine rudimentäre Fehlerbehandlung aufweisen und bei Problemen die Abweichung vom erwarteten Token zum tatsächlich eingelesenen Token ausgeben. Darüber hinaus braucht der Parser keinen Recovery-Modus haben o.ä., d.h. nach der Fehlermeldung darf Ihr Parser “aussteigen”.

4.5 A4.5: AST (1P)

Definieren Sie einen AST für die Zielsprache. Welche Informationen aus dem Eingabeprogramm müssen repräsentiert werden?

Programmieren Sie die entsprechenden Datenstrukturen in Java.

Programmieren Sie außerdem eine Traversierung des Parse-Trees, die den AST erzeugt. Testen Sie dies mit Ihren in der ersten Aufgabe entwickelten Beispielprogrammen.

4.6 A4.6: Recherche und Diskussion (1P)

Recherchieren Sie, welche Open-Source-Projekte auf handgeschriebene *recursive descent* Parser setzen (oder umgestiegen sind) und welche Gründe es dafür in diesen Projekten gibt. Was spricht aus Ihrer persönlichen Sicht für und gegen ANTLR?

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** f15e08b (homework: finalize B04 (#376), 2025-10-03)